Tomás Alarcón Martínez Troncoso - 202420126
Juan José Scovino Villarreal - 202421005
Samuel Uribe - 202421501

DPOO Documento Proyecto 3

Introducción

Este documento presenta el diseño completo del Sistema de Venta de Tiquetes para Eventos, una plataforma de ticketing orientada a objetos desarrollada en Java que permite la gestión integral de eventos, venta de tiquetes, operaciones del marketplace de reventa, administración financiera y sistema de impresión de tiquetes con códigos QR dinámicos. En esta tercera entrega, el sistema ha evolucionado desde una interfaz de consola a una interfaz gráfica completa basada en Swing, incorporando nuevas funcionalidades clave como la impresión de tiquetes con códigos QR dinámicos y bloqueo de reimpresión. Se mantienen todas las funcionalidades del proyecto anterior y se añaden las requeridas para esta fase.
El sistema permite:

- Clientes:
  - Comprar tiquetes y recibir cargos asociados.
  - Visualizar, almacenar y transferir tiquetes.
  - Publicar tiquetes en el Marketplace para reventa.
  - Enviar contraofertas y aceptar compras.
  - Consultar reportes personales (tiquetes, saldo, reventas).
  - Administrar saldo virtual y transacciones.
  - Imprimir tiquetes con código QR dinámico. Previsualizar tiquetes antes de imprimir.
- Organizadores:
  - Crear eventos y asociarlos a un venue aprobado.
  - Crear y administrar las localidades del evento.
  - Consultar ventas, ocupación y reportes financieros.
  - Proponer venues y gestionar disponibilidad.
  - Administrar ofertas especiales del evento.
- Administradores:
  - Aprobar o rechazar venues nuevos.
  - Configurar cargos y tarifas del sistema.
  - Cancelar eventos y aprobar reembolsos.
  - Moderar el Marketplace y resolver conflictos.
  - Generar y consultar reportes financieros globales.
- Persistencia:
  - Almacenamiento en archivos planos para mantener la información entre sesiones, extendido para soportar el Marketplace y el nuevo sistema de impresión.
- Interfaz Gráfica:
  - Navegación intuitiva por pestañas y ventanas.
  - Diálogos de impresión de tiquetes con previsualización.

    - Generación dinámica de códigos QR para cada tiquete.
    - Bloqueo de reimpresión de tiquetes ya impresos.
    - Sistema de auditoría para seguimiento de impresiones.

Actores del Sistema

- Cliente (Comprador de tiquetes): El Cliente es el usuario final que adquiere tiquetes dentro de la plataforma. Puede visualizar sus tiquetes, almacenarlos, transferirlos a otros usuarios y gestionar su saldo virtual. Además, participa activamente en el Marketplace publicando tiquetes para reventa, enviando contraofertas, aceptando compras y revisando sus estadísticas de actividad. El cliente también puede consultar reportes personales que incluyen su historial de compras, tiquetes revendibles y saldo disponible. En el proyecto, el cliente puede imprimir sus tiquetes con un código QR único que contiene información del evento y del tiquete, y que una vez impreso no puede volver a imprimirse, transferirse ni revenderse. Representa el actor principal del proceso de venta y circulación de tiquetes.
- Organizador (Promotor de eventos): El Organizador es el responsable de crear y administrar eventos dentro del sistema. Puede registrar eventos nuevos, configurar y modificar localidades, supervisar la disponibilidad de tiquetes y gestionar las ofertas relacionadas con sus eventos. También puede acceder a reportes financieros asociados a sus eventos, visualizar transacciones y hacer seguimiento al rendimiento de ventas. Este actor es clave para el suministro de eventos y tiquetes dentro de la plataforma.
- Administrador (Gestor del sistema): El Administrador controla las funciones globales de la plataforma. Tiene la capacidad de configurar cargos y tarifas del sistema, aprobar o rechazar venues propuestos por organizadores, cancelar eventos activos y procesar reembolsos a clientes. También supervisa el funcionamiento general del Marketplace, gestiona reportes financieros y asegura la integridad de la operación mediante la administración de usuarios, logs y transacciones. Es la autoridad máxima del sistema y garantiza su correcto funcionamiento.

Visión General del Sistema

El sistema emula el funcionamiento de una plataforma de venta de tiquetes similar a Ticketmaster o Tuboleta. Permite administrar eventos, vender distintos tipos de tiquetes, procesar transacciones, gestionar reventas en un Marketplace interno y mantener registros financieros y operativos.
El sistema soporta:

- Múltiples tipos de eventos: musicales, culturales, deportivos, ferias, religiosos, conferencias, etc.
- Diversos tipos de tiquetes: básicos, numerados, pases de temporada, tiquetes múltiples, y paquetes deluxe con beneficios incluidos.
- Marketplace interno para reventa de tiquetes entre usuarios con contraofertas, negociación y transferencias controladas.
- Gestión financiera integral, con cargos configurables, reportes detallados por organizador, administración de reembolsos y registro de transacciones.
- Persistencia en archivos planos, garantizando que usuarios, eventos, transacciones, tiquetes y marketplace permanezcan almacenados entre sesiones.

- Roles separados (Cliente, Organizador, Administrador) con capacidades específicas y controladas de acuerdo con el flujo real del sistema.
- Interfaz gráfica de usuario (GUI) basada en Swing:
    - Navegación por pestañas para cada rol de usuario.
    - Tablas interactivas para visualización de eventos, tiquetes, ofertas, etc.
    - Diálogos de impresión de tiquetes con previsualización y código QR.
    - Generación dinámica de códigos QR para cada tiquete, conteniendo información única (evento, ID, fechas).
    - Bloqueo de reimpresión: un tiquete impreso no puede volver a imprimirse, transferirse ni revenderse.
    - Sistema de impresión nativo de Java con previsualización.
- Sistema de impresión con código QR:
    - Cada tiquete genera un código QR único con datos específicos.
    - El código QR contiene: nombre del evento, ID del tiquete, fecha del evento, fecha de impresión.
    - Validación de integridad mediante formato estructurado.
    - Compatibilidad con lectores de QR estándar.

El sistema está diseñado con una arquitectura orientada a objetos, con múltiples módulos colaborando entre sí, respetando principios de encapsulación, composición, agregación y herencia.

Características Principales

- Gestión de Eventos:
    - Los organizadores pueden crear eventos, asignar fechas, categorías, venues y parámetros generales.
    - Cada evento puede asociarse a un venue físico con aforos y localidades específicas.
    - Configuración de localidades, cada una con precio base, capacidad y políticas (revendible o no).
    - Gestión de tiquetes disponibles, precios finales, cargos adicionales y estados del evento.
    - Visualización de eventos activos para los clientes desde la interfaz gráfica.
- Sistema de Tiquetes:
    - Tiquete Basico: Acceso estándar a una localidad con cargos básicos.
    - TiqueteNumerado: Incluye asiento específico, fila, número y control de duplicidad.
    - Tiquete Múltiple: Permite agrupar varios tiquetes en una sola compra.
    - PaseTemporada: Da acceso a varios eventos dentro de un rango de fechas, usando una lista interna de eventos asociados.
    - TiqueteDeluxe: Un paquete especial que puede contener:
        - Varios tiquetes internos (composición), beneficios extra (bebidas, accesos VIP, merch, etc.), cargos y valores adicionales.
    - Nuevo estado "impreso": Los tiquetes pueden marcarse como impresos, bloqueando reimpresión y transferencia.
- Transacciones y Gestión Financiera:
    - Compras a través de saldo virtual del cliente.
    - Cálculo automático de cargos, comisiones y tarifas.

    - Reembolsos por cancelaciones o causas justificadas.
    - Reportes financieros por organizador o administrador.
- Marketplace en Reventa:
    - El Marketplace gestiona formalmente:
        - Ofertas activas (con agregación)
        - Contraofertas pendientes
        - Transacciones entre clientes
        - Historias de actividad mediante logs
    - En este Marketplace interno los clientes pueden:
        - Publicar tiquetes para reventa con precio personalizado.
        - Recibir y enviar contraofertas.
        - Aceptar o rechazar ofertas de otros usuarios.
        - Transferir automáticamente el tiquete al comprador cuando la transacción es aprobada.
        - Ver estadísticas personales y globales (ofertas publicadas, ventas, contraofertas recibidas).
        - Registrar cada oferta y contraoferta con un LogRegistry.
- Interfaz Gráfica y Sistema de Impresión:
    - Interfaz gráfica basada en Swing con ventanas y diálogos.
    - Navegación por pestañas para cada rol de usuario (Cliente, Organizador, Administrador).
    - Diálogo de impresión de tiquetes que muestra una previsualización con información del evento y código QR.
    - Generación dinámica de códigos QR para cada tiquete, conteniendo:
        - Nombre del evento
        - ID del tiquete
        - Fecha del evento
        - Fecha de impresión (obtenida en tiempo real)
    - Bloqueo de reimpresión: una vez impreso un tiquete, su estado cambia a "impreso" y no puede volver a imprimirse, transferirse ni revenderse.
    - Integración con el sistema de impresión nativo de Java (PrinterJob, Printable).

Arquitectura del Sistema

El sistema implementa una arquitectura modular orientada a objetos, organizada en paquetes que separan claramente la presentación, la lógica de negocio, las entidades del dominio y la persistencia de datos. La interacción del usuario se realiza a través de una interfaz gráfica basada en Swing, organizada en un módulo principal (MainGUI) y paneles específicos para cada rol (ClientePanel, OrganizadorPanel, AdministradorPanel), que reemplazan las consolas del anterior proyecto.
La comunicación entre la interfaz gráfica y el núcleo del sistema se realiza a través de la clase SistemaVentaTiquetes, que actúa como fachada central del sistema. Esta fachada coordina los subsistemas principales: usuarios, eventos, tiquetes, marketplace, finanzas y persistencia. Esta separación controla el flujo del sistema y mantiene bajo acoplamiento entre capas. La persistencia se realiza mediante archivos planos ubicados en la carpeta ./datos_sistema, donde se almacenan usuarios, eventos, venues, transacciones, reembolsos, ofertas del Marketplace y ahora también los logs del Marketplace.

El diseño global sigue una arquitectura en capas, donde cada paquete tiene responsabilidades bien definidas:

- Capa de Presentación (GUI):
    - MainGUI: Ventana principal que gestiona la navegación entre paneles usando CardLayout.
    - LoginPanel: Panel de inicio de sesión para todos los tipos de usuarios.
    - UserPanel (abstracta): Panel base con funcionalidades comunes para todos los paneles de usuario.
    - ClientePanel, OrganizadorPanel, AdministradorPanel: Paneles específicos para cada rol, que extienden UserPanel.
    - ImpresionTiqueteDialog: Diálogo para previsualizar e imprimir tiquetes con código QR.
    - SimpleQRGenerator: Clase utilitaria para generar códigos QR dinámicos usando la librería ZXing.
- Capa de Lógica de Negocio (sistema y usuarios):
    - Contiene la clase SistemaVentaTiquetes y las clases Usuario, Cliente, Organizador y Administrador, responsables de orquestar las operaciones del sistema.
- Capa de Dominio (eventos, tiquetes, marketplace, finanzas):
    - Incluye la representación de eventos, localidades, tipos de tiquetes, marketplace de reventa, transacciones y reembolsos.
- Capa de Persistencia:
    - Gestionada por GestorPersistencia, controla lectura/escritura en archivos y garantiza que el estado del sistema se mantenga entre sesiones.
    - Extendida para soportar el Marketplace y los logs.

La comunicación entre capas ocurre únicamente a través de SistemaVentaTiquetes, lo que asegura separación de responsabilidades, facilidad de mantenimiento y una futura integración sencilla con otras interfaces sin alterar la lógica interna.

Estructura de Paquetes:

```
src/
├── gui/
│   ├── MainGUI.java
│   ├── LoginPanel.java
│   ├── UserPanel.java
│   ├── ClientePanel.java
│   ├── OrganizadorPanel.java
│   ├── AdministradorPanel.java
│   ├── ImpresionTiqueteDialog.java
│   ├── MusicPlayer.java
│   └── qr/
│       └── SimpleQRGenerator.java
│
├── usuarios/
│   ├── Usuario.java (abstracta)
│   ├── Cliente.java
│   ├── Organizador.java
```

```
│   └── Administrador.java
│
├── eventos/
│   ├── Evento.java
│   ├── Localidad.java
│   ├── Venue.java
│   └── Oferta.java
│
├── tiquetes/
│   ├── Tiquete.java (abstracta)
│   ├── TiqueteBasico.java
│   ├── TiqueteNumerado.java
│   ├── TiqueteMultiple.java (abstracta)
│   ├── PaseTemporada.java
│   ├── PaqueteDeluxe.java
│   └── Beneficio.java
│
├── finanzas/
│   ├── ConfiguracionCargos.java
│   ├── Transaccion.java
│   ├── Rembolso.java
│   └── ReporteFinanciero.java
│
├── sistema/
│   └── SistemaVentaTiquetes.java
│
├── marketplace/
│   ├── ContraOferta.java
│   ├── LogEntry.java
│   ├── LogRegistry.java
│   ├── Marketplace.java
│   └── OfertaReventa.java
│
└── persistencia/
    ├── IPersistencia.java (interfaz)
    └── GestorPersistencia.java
```

Diagramas de Clases:

Los diagramas de clases han sido actualizados para incluir las nuevas clases de la interfaz gráfica y las modificaciones en el sistema de tiquetes para soportar la impresión con QR. A continuación, se describe brevemente los cambios principales; los diagramas completos se encuentran en el archivo PDF adjunto.

- Diagrama de Clases Completo
    - Clases de la GUI: MainGUI, LoginPanel, UserPanel, ClientePanel, OrganizadorPanel, AdministradorPanel, ImpresionTiqueteDialog, SimpleQRGenerator, MusicPlayer.

    - Relaciones de herencia: UserPanel es una clase abstracta de la que heredan los paneles específicos de cada rol.
    - Relaciones de asociación: Los paneles de usuario se asocian con las clases de dominio a través de SistemaVentaTiquetes.
    - Nuevos métodos en Tiquete: Para soportar la impresión y el bloqueo (por ejemplo, puedeImprimir(), marcarComoImpreso()).
    - Integración de ZXing: La clase SimpleQRGenerator utiliza la librería ZXing para generar códigos QR.
- Diagrama de Clases de Alto Nivel
    - Se mantiene la arquitectura en capas, pero ahora la capa de presentación está representada por el paquete gui en lugar de las consolas.
    - La capa de negocio y la de dominio permanecen sin cambios estructurales.
- Diagrama de Clases de la Interfaz Gráfica
    - MainGUI contiene un CardLayout para cambiar entre paneles.
    - LoginPanel permite el inicio de sesión y registro.
    - UserPanel define la estructura común (encabezado, pestañas, botón de cerrar sesión).
    - Los paneles específicos (ClientePanel, etc.) contienen tablas y botones para las funcionalidades de cada rol.
    - ImpresionTiqueteDialog se abre desde ClientePanel y muestra un diálogo modal con la previsualización del tiquete y el código QR.

Decisiones de Diseño Clave

- Transición de Consola a Interfaz Gráfica:
    - Decisión: Reemplazar las consolas (MainCliente, MainOrganizador, MainAdministrador) por una interfaz gráfica basada en Swing.
    - Justificación: Cumplir con los requisitos del proyecto 3, mejorar la usabilidad y proporcionar una experiencia más intuitiva para el usuario.
- Navegación por Pestañas:
    - Decisión: Implementar JTabbedPane en cada panel de usuario para organizar las funcionalidades.
    - Justificación: Agrupar funcionalidades relacionadas, reducir la complejidad cognitiva y facilitar la expansión futura.
- Generación Dinámica de Códigos QR:
    - Decisión: Usar la librería ZXing para generar códigos QR únicos por tiquete.
    - Justificación: Cumplir con el requisito de códigos dinámicos que contengan información específica del tiquete (evento, ID, fechas) y que sean legibles por cualquier lector estándar.
- Mecanismo de Bloqueo de Reimpresión:
    - Decisión: Cambiar el estado del tiquete a "impreso" después de la primera impresión y validar este estado antes de permitir una nueva impresión, transferencia o reventa.
    - Justificación: Garantizar que un tiquete físico no pueda ser duplicado, aumentando la seguridad del sistema.
- Diálogo de Impresión con Previsualización:
    - Decisión: Implementar un diálogo modal que muestre una previsualización del tiquete antes de imprimir.

    - Justificación: Permitir al usuario verificar la información y el código QR antes de imprimir, reduciendo errores.
- Persistencia Automática:
    - Decisión: Guardar automáticamente los datos después de operaciones críticas (compra, reventa, impresión, etc.).
    - Justificación: Minimizar la pérdida de datos y mantener la consistencia del sistema.
- Música de Fondo:
    - Decisión: Incluir un reproductor de música (MusicPlayer) que reproduce una canción en bucle durante el uso de la aplicación.
    - Justificación: Mejorar la experiencia del usuario, aunque es un elemento opcional y no crítico.
- Mantenimiento de la Arquitectura Existente:
    - Decisión: No modificar la lógica de negocio ni las clases de dominio; solo adaptar la capa de presentación.
    - Justificación: Aprovechar el diseño sólido del proyecto 2, demostrando que la arquitectura permite cambiar la interfaz sin afectar el núcleo.

Patrones de Diseño Aplicados

- Facade:
    - Dónde: SistemaVentaTiquetes actúa como fachada/coordinador de operaciones.
    - Por qué: Simplifica la interacción de la GUI con la lógica de negocio.
- DAO / Repository:
    - Dónde: IPersistencia + GestorPersistencia.
    - Por qué: Separar acceso a almacenamiento (archivos) de lógica del dominio.
- Composite / Aggregation:
    - Dónde: PaqueteDeluxe contiene Tiquete(s) y Beneficio(s) - composición.
    - Por qué: Modela objetos compuestos con ciclo de vida dependiente.
- Strategy:
    - Propósito: Abstracción del cálculo de precio final (política de descuentos, paquetes, recargos).
    - Por qué: Facilita añadir nuevas estrategias de precio sin alterar la jerarquía de tiquetes.
- Template Method:
    - Dónde: UserPanel define la estructura común (método createHeader, createTabbedPane) y deja que las subclases implementen los detalles.
    - Por qué: Reutilizar código común en los paneles de cada rol.
- Observer (implícito en Swing):
    - Dónde: Los listeners de eventos en botones y tablas.
    - Por qué: Responder a las acciones del usuario de manera desacoplada.

Flujos de Trabajo Principales

- Flujo de Autenticación y Navegación en GUI
    - Usuario ingresa credenciales en LoginPanel
    - Sistema valida usuario mediante SistemaVentaTiquetes.autenticar()

    - Si autenticación es exitosa, MainGUI redirige al panel correspondiente:
        - Cliente → ClientePanel
        - Organizador → OrganizadorPanel
        - Administrador → AdministradorPanel
    - Panel específico carga datos iniciales y muestra interfaz personalizada
    - Usuario navega mediante pestañas para acceder a diferentes funcionalidades
    - Botón "Cerrar Sesión" retorna a pantalla de login
- Flujo de Compra de Tiquetes (Cliente)
    - Cliente selecciona evento de tabla en pestaña "Eventos Disponibles"
    - Hace clic en botón "Comprar Tiquete"
    - Sistema verifica: }
        - Disponibilidad en localidad seleccionada
        - Saldo suficiente del cliente
        - Validez del evento (activo, fecha futura)
    - Si verificación es exitosa:
        - Se crea objeto TiqueteBasico asociado al cliente
        - Se deduce saldo del cliente
        - Se actualiza contador de tiquetes vendidos en localidad
        - Se registra transacción Se guardan datos automáticamente
    - Se muestra confirmación y se actualiza tabla "Mis Tiquetes"
- Flujo de Impresión de Tiquete con QR
    - Cliente selecciona tiquete de tabla "Mis Tiquetes"
    - Hace clic en botón "Imprimir Tiquete"
    - Se abre ImpresionTiqueteDialog (diálogo modal) con tiquete seleccionado
    - Sistema verifica que tiquete NO tenga estado "impreso"
    - Diálogo genera datos para QR usando SimpleQRGenerator:
        - Nombre del evento
        - ID del tiquete
        - Fecha del evento
        - Fecha de impresión (actual)
    - Se muestra previsualización con información del tiquete y código QR
    - Cliente hace clic en "Imprimir"
    - Sistema cambia estado del tiquete a "impreso"
    - Se abre diálogo de impresión nativo de Java (PrinterJob)
    - Si usuario confirma, se imprime tiquete con QR
    - Se guardan datos automáticamente
    - Tiquete queda bloqueado para reimpresión, transferencia y reventa
- Flujo de Publicación de Oferta en Marketplace
    - Cliente selecciona tiquete de "Mis Tiquetes"
    - Hace clic en "Publicar en Marketplace"
    - Sistema verifica que tiquete sea revendible (no impreso, no usado, evento futuro)
    - Diálogo solicita precio de venta Si precio es válido:
        - Se crea OfertaReventa asociada al tiquete y cliente
        - Se agrega al Marketplace
        - Se registra acción en LogRegistry
        - Se guardan datos automáticamente

    - Oferta aparece en pestaña "Marketplace" para otros usuarios
- Flujo de Contraoferta en Marketplace
    - Cliente navega a pestaña "Marketplace"
    - Selecciona oferta de otro usuario
    - Hace clic en "Hacer Contraoferta"
    - Diálogo solicita precio propuesto
    - Sistema crea ContraOferta asociada a OfertaReventa y cliente comprador
    - Se agrega a lista de contraofertas pendientes del Marketplace
    - Se registra acción en logs
    - Se guardan datos automáticamente
    - Contraoferta queda pendiente de respuesta del vendedor
- Flujo de Aceptación de Contraoferta
    - Vendedor accede a lista de contraofertas pendientes
    - Selecciona contraoferta específica
    - Hace clic en "Aceptar Contraoferta"
    - Sistema verifica:
        - Contraoferta en estado "PENDIENTE"
        - Comprador tiene saldo suficiente
        - Tiquete sigue siendo revendible
    - Si verificación es exitosa:
        - Se transfiere tiquete a comprador (cambia dueño)
        - Se transfiere pago (deduce saldo comprador, recarga vendedor)
        - Marca oferta como inactiva y contraoferta como "ACEPTADA"
        - Registra acción en logs
        - Guarda datos automáticamente
    - Ambos clientes reciben confirmación
- Flujo de Creación de Evento (Organizador)
    - Organizador navega a pestaña "Dashboard" o "Mis Eventos"
    - Hace clic en "Crear Evento"
    - Diálogo solicita:
        - ID del evento
        - Nombre
        - Fecha
        - Tipo
        - Venue (de lista de venues aprobados)
    - Sistema valida:
        - Venue está aprobado
        - No hay conflicto de fecha en el venue
    - Si es válido:
        - Crea nuevo Evento asociado al organizador y venue
        - Agrega a lista de eventos del sistema
        - Guarda datos automáticamente
    - Evento aparece en tabla de eventos del organizador
- Flujo de Aprobación de Venue (Administrador)
    - Administrador navega a pestaña "Venues Pendientes"
    - Selecciona venue pendiente de tabla
    - Hace clic en "Aprobar Venue"
    - Sistema cambia estado del venue a aprobado (setAprobado(true))

    - Guarda datos automáticamente
    - Venue aparece en lista de venues aprobados para uso de organizadores
- Flujo de Configuración de Cargos (Administrador)
    - Administrador navega a pestaña "Configuración"
    - Ajusta valores de cargos (porcentajes) y cuota de emisión
    - Hace clic en "Actualizar Cargos"
    - Sistema actualiza instancia de ConfiguracionCargos
    - Guarda datos automáticamente
    - Nuevos cargos aplican a futuras transacciones
- Flujo de Cierre de Sesión
    - Usuario hace clic en "Cerrar Sesión" en cualquier panel
    - Sistema guarda datos automáticamente (si no se ha hecho)
    - MainGUI muestra LoginPanel nuevamente
    - Usuario puede iniciar sesión con otra cuenta o cerrar aplicación

Modelo de Persistencia

El sistema utiliza archivos planos como medio de persistencia, extendido para soportar el Marketplace y ahora también para almacenar el estado de impresión de los tiquetes (a través del estado del tiquete). La estructura de carpetas y archivos es la siguiente:

```
/datos_sistema/
├── usuarios.txt
├── venues.txt
├── eventos.txt
├── transacciones.txt
├── reembolsos.txt
├── ofertasReventa.txt
├── contraofertas.txt
└── logMarketplace.txt
```

Los formatos de los archivos son los mismos que en el proyecto 2, excepto que en usuarios.txt y eventos.txt se incluyen los nuevos estados (como "impreso" para tiquetes). No se requieren archivos adicionales porque el estado de impresión se almacena en el tiquete, y los tiquetes están asociados a los clientes (que se guardan en usuarios.txt).

Validaciones y Restricciones

Se mantienen todas las validaciones del proyecto 2, y se añaden:

- Impresión de tiquetes:
    - Un tiquete solo puede imprimirse si su estado no es "impreso".
    - Después de imprimir, el estado cambia a "impreso" y no puede revertirse.
    - Un tiquete impreso no puede transferirse ni revenderse.
- Códigos QR:
    - Deben generarse dinámicamente para cada tiquete, conteniendo al menos: nombre del evento, ID del tiquete, fecha del evento, fecha de impresión.
    - El código QR debe ser legible por lectores estándar.
- Interfaz gráfica:

    - Los campos de entrada deben validarse (por ejemplo, precios positivos, fechas válidas).
    - Las tablas deben actualizarse después de operaciones críticas.

Casos de Uso Principales / Historias de Usuario

Se añaden nuevas historias de usuario relacionadas con la interfaz gráfica y la impresión:

- USUARIO CLIENTE:
    - HU-014: Impresión de Tiquetes con QR
        - Como cliente que ha comprado tiquetes
        - Quiero poder imprimir mis tiquetes con un código QR único
        - Para tener una copia física y poder ingresar al evento
        - Criterios de Aceptación:
            - Debo ver un botón "Imprimir" en mis tiquetes.
            - Al imprimir, debo ver una previsualización con la información del tiquete y un código QR.
            - El código QR debe contener información específica del tiquete (evento, ID, fechas).
            - Una vez impreso, el tiquete no debe poder imprimirse nuevamente.
            - El tiquete impreso no debe poder transferirse ni revenderse.
    - HU-015: Navegación Intuitiva en Interfaz Gráfica
        - Como cliente
        - Quiero una interfaz gráfica con pestañas y tablas
        - Para poder gestionar mis tiquetes y compras de manera visual
        - Criterios de Aceptación:
            - Debo ver una ventana con pestañas para "Mi Perfil", "Mis Tiquetes", "Eventos Disponibles", "Marketplace".
            - Las tablas deben mostrar información clara y actualizarse después de cada operación.
            - Debo poder cerrar sesión y volver al inicio de sesión.
- USUARIO ORGANIZADOR:
    - HU-016: Gestión de Eventos en Interfaz Gráfica
        - Como organizador
        - Quiero gestionar mis eventos a través de una interfaz gráfica
        - Para crear y modificar eventos de manera visual
        - Criterios de Aceptación:
            - Debo ver una ventana con pestañas para "Dashboard", "Mis Eventos", "Venues Disponibles", "Reportes".
            - Debo poder crear eventos mediante un formulario.
            - Debo poder agregar localidades a mis eventos.
- USUARIO ADMINISTRADOR:
    - HU-017: Moderación y Reportes en Interfaz Gráfica
        - Como administrador
        - Quiero supervisar el sistema a través de una interfaz gráfica
        - Para aprobar venues, moderar el marketplace y ver reportes financieros
        - Criterios de Aceptación:

                - Debo ver una ventana con pestañas para "Dashboard", "Venues Pendientes", "Gestión de Eventos", "Moderar Marketplace", "Configuración".
                - Debo poder aprobar o rechazar venues desde una tabla.
                - Debo poder ver logs del marketplace en un área de texto.
- REQUISITOS DEL SISTEMA:
    - HU-018: Persistencia y Carga de Datos en GUI
        - Como usuario del sistema
        - Quiero que la interfaz gráfica cargue y guarde los datos automáticamente
        - Para que mis cambios se mantengan entre sesiones sin esfuerzo
        - Criterios de Aceptación:
            - Al iniciar la aplicación, la GUI debe cargar los datos desde los archivos.
            - Después de operaciones críticas (compra, impresión, reventa), los datos deben guardarse automáticamente.
            - Al cerrar la aplicación, los datos deben guardarse.

Conclusiones

El sistema BoletaMaster ha evolucionado exitosamente desde una aplicación de consola a una interfaz gráfica completa basada en Swing, cumpliendo con todos los requisitos funcionales y no funcionales. La arquitectura modular permitió que la transición fuera suave, sin modificar la lógica de negocio ni las clases de dominio.

Principales logros del proyecto 3:

- Interfaz gráfica intuitiva con navegación por pestañas para cada rol.
- Sistema de impresión de tiquetes con previsualización y código QR dinámico.
- Bloqueo de reimpresión, transferencia y reventa de tiquetes impresos.
- Integración de la librería ZXing para generación de códigos QR.
- Persistencia automática después de operaciones críticas.
- Mantenimiento de todas las funcionalidades del proyecto #2 (marketplace, finanzas, etc.).

Extensibilidad y Futuras Mejoras:

- Integración con APIs de pago en línea.
- Soporte para múltiples idiomas.
- Mejoras en la seguridad (encriptación de contraseñas, autenticación de dos factores).
- Aplicación móvil o versión web.

En conjunto, el sistema demuestra un diseño sólido, modular y escalable, apto para su uso en entornos reales de ticketing.